MEDIQUERY

Note méthodologique

Conception d'un assistant médical fondé sur le Retrieval-Augmented Generation (RAG)

Sambou Diao · Data Scientist / GenAI

AVERTISSEMENT — Prototype pédagogique non validé cliniquement. MediQuery ne fournit pas de diagnostic, ne prescrit pas de traitement et ne remplace pas un professionnel de santé.

1. Objectif et périmètre

MediQuery est un prototype d'assistant conversationnel qui permet d'interroger en français un corpus documentaire médical. Il combine une recherche documentaire et un modèle de langage afin de produire une réponse à partir des passages retrouvés. Le projet illustre la conception d'un pipeline RAG modulaire, testable et attentif à la traçabilité des réponses ainsi qu'au coût d'inférence.

Le corpus de démonstration s'appuie sur le guide de la Haute Autorité de santé consacré au parcours de soins du patient adulte vivant avec un diabète de type 2. La couverture de l'assistant est limitée au contenu effectivement indexé.

MediQuery est un projet pédagogique non validé cliniquement. Il ne fournit pas de diagnostic, ne prescrit pas de traitement et ne remplace pas un professionnel de santé.

2. Vue d'ensemble de l'architecture

Flux principal : PDF → extraction du texte → découpage en segments → embeddings et indexation → recherche hybride → construction du contexte → génération avec Claude → réponse et références.

L'application est structurée en composants distincts : ingestion, récupération documentaire, génération, orchestration et interface Streamlit. Cette séparation facilite les tests, le diagnostic des erreurs et l'évolution indépendante des composants.

DOCUMENT PDF → EXTRACTION & CHUNKING → EMBEDDINGS / CHROMADB
QUESTION → RECHERCHE VECTORIELLE + BM25 → FUSION RRF
→ CONTEXTE DOCUMENTAIRE → CLAUDE API → RÉPONSE + RÉFÉRENCES

3. Ingestion et représentation documentaire

3.1 Extraction et découpage

Le PDF est chargé et transformé en documents textuels. Le texte est segmenté en chunks avec chevauchement afin de limiter la perte de contexte aux frontières des découpages. Les métadonnées, notamment la source et la page lorsqu'elles sont disponibles, sont conservées pour rattacher les passages à leur origine.

3.2 Embeddings et stockage

Le modèle configuré est sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. Chaque chunk est représenté par un vecteur destiné à la recherche sémantique. Les vecteurs et les documents associés sont stockés dans ChromaDB.

L'index est construit lors de l'ingestion. Il convient de le réutiliser tant que le corpus, les paramètres de découpage et le modèle d'embeddings restent compatibles, plutôt que de recalculer les embeddings à chaque lancement.

4. Recherche hybride

La récupération combine deux mécanismes complémentaires. La recherche vectorielle vise la proximité sémantique entre la question et les chunks. BM25 classe les passages selon leurs correspondances lexicales. Les classements sont ensuite fusionnés par Reciprocal Rank Fusion (RRF).

La stratégie hybride sans reranking a été retenue après une première évaluation exploratoire. Le reranking a été testé, mais n'a pas montré de gain uniforme sur les métriques observées. Ce choix devra être réévalué avec un jeu de questions plus large.

5. Génération et ancrage documentaire

Les chunks récupérés sont formatés en contexte et transmis à Claude via l'API Anthropic. Le prompt système demande au modèle de répondre en français, de s'appuyer sur les informations fournies, de ne pas inventer de faits et de signaler lorsque le contexte ne permet pas de répondre de façon fiable.

La réponse est associée aux références documentaires issues de la récupération. Ces références facilitent la vérification, mais ne garantissent pas à elles seules que chaque affirmation est correctement étayée.

Le mécanisme d'abstention s'appuie sur l'absence de contexte et sur la détection de formulations de refus ou d'insuffisance dans la réponse générée. Cette détection heuristique constitue une protection partielle ; elle ne remplace pas un classificateur de périmètre ni une validation systématique.

6. Évaluation exploratoire

Une première comparaison a été effectuée sur un jeu limité de cinq questions, à l'aide de Recall@5 et de Mean Reciprocal Rank (MRR@5).

Stratégie	Recall@5	MRR@5
Recherche hybride	0,700	0,750
Hybride + reranking	0,600	0,767

Sur ce jeu exploratoire, la recherche hybride sans reranking récupère une plus grande proportion des passages pertinents dans le top 5, tandis que le reranking présente un MRR légèrement supérieur. Cinq questions ne permettent pas de conclure à une supériorité générale : ces résultats sont indicatifs et sensibles au choix des questions.

Une évaluation plus robuste devrait inclure des questions représentatives, des passages de référence, la qualité des réponses, la fidélité au contexte, la justesse des citations, le comportement d'abstention, la latence et le coût par requête.

7. Maîtrise des coûts

La génération via API engendre un coût dépendant du modèle et des volumes de tokens en entrée et en sortie. MediQuery privilégie une approche sans mémoire conversationnelle transmise au modèle : l'historique peut rester visible dans l'interface, mais il n'est pas ajouté systématiquement aux requêtes Claude.

- Limiter le nombre de chunks transmis au LLM et contrôler la taille du contexte.
- Fixer une limite raisonnable au nombre de tokens générés.
- Éviter les appels API lorsque la question est vide ou qu'aucun contexte n'a été retrouvé.
- Réutiliser l'index ChromaDB plutôt que de réingérer le corpus à chaque lancement.
- Mesurer les tokens et les coûts observés avant de fixer un coût moyen par question.

Aucun coût moyen par question n'est affirmé ici : il doit être calculé à partir des consommations réelles et des tarifs applicables au modèle.

8. Qualité logicielle et reproductibilité

Le projet comprend des tests unitaires et des tests d'intégration pour vérifier le comportement des composants du pipeline. Commandes de référence :

```
pytest -q
ruff check .
```

La configuration est centralisée et les secrets sont fournis par variables d'environnement. Le fichier .env ne doit pas être versionné ; seul un fichier .env.example dépourvu de secrets doit être publié.

9. Limites, sécurité et confidentialité

- La réponse dépend de la couverture et de la qualité du corpus indexé.
- Un passage proche de la question peut être insuffisant pour y répondre.
- Les prompts et la récupération réduisent certains risques d'hallucination sans les éliminer.
- Les références doivent être vérifiées au regard des affirmations produites.
- Les données envoyées à l'API Anthropic sont soumises aux conditions et règles de traitement du fournisseur.
- Aucune donnée personnelle ou donnée de santé identifiante ne doit être utilisée dans cette démonstration.
- L'application n'est ni un dispositif médical validé ni un outil d'aide à la décision clinique.

10. Références et évolutions possibles

Document de démonstration : Haute Autorité de santé, « Parcours de soins du patient adulte vivant avec un diabète de type 2 ».

Lien officiel : https://www.has-sante.fr/upload/docs/application/pdf/2025-07/parcours_de_soins_du_patient_adulte_vivant_avec_un_diabete_de_type_2_-_guide_2025-07-15_16-42-8_797.pdf

- Élargir et versionner le jeu de questions d'évaluation.
- Évaluer la fidélité des réponses et la qualité des citations.
- Renforcer la détection des questions hors périmètre.
- Suivre la latence, les tokens et le coût par requête.
- Ajouter des tests de non-régression sur des cas médicaux sensibles.

MediQuery — prototype pédagogique · Sambou Diao